Week 2 Assignment

NumPy & Pandas for Data Analytics

Practical Report: Source Code, Outputs, Explanations & Insights

Submitted by	Deepanshu
Total Marks	100
Tools used	Python 3, NumPy, Pandas
Google Drive link	PASTE YOUR LINK HERE (set access to 'Anyone with the link can view')

Contents & Marks Breakdown

#	Task	Marks	Script file
1	NumPy Introduction & Arrays	10	task01_numpy_arrays.py
2	NumPy Indexing, Slicing & Reshaping	10	task02_indexing_slicing_reshaping.py
3	NumPy Mathematical & Statistical Operations	10	task03_numpy_math_stats.py
4	Pandas Series & DataFrame	10	task04_pandas_series_dataframe.py
5	Reading & Inspecting Data	10	task05_read_inspect.py
6	Selecting, Filtering & Sorting Data	10	task06_select_filter_sort.py
7	Handling Missing Values	10	task07_missing_values.py
8	Merge, Concatenate, GroupBy & Pivot Table	15	task08_merge_groupby_pivot.py
9	Exporting Data	5	task09_export.py
10	Mini Data Analysis Project - Sales Performance	10	task10_mini_project.py

Datasets Used

Tasks 5-10 use two CSV files stored in the **data/** folder. They were generated with *make_datasets.py* (fixed random seed, so results are reproducible). The sales file intentionally contains missing values so that Task 7 has real gaps to handle. Tasks 1-4 use small arrays and DataFrames created inside the scripts.

sales_data.csv - 200 rows x 10 columns

order_id, order_date, customer_id, product, category, quantity, unit_price, discount, region, payment_mode. Missing values: quantity (12), unit_price (10), discount (10), region (8).

```
order_id,order_date,customer_id,product,category,quantity,unit_price,discount,region,payment_mode
1001,2026-01-01,24,T-Shirt,Clothing,2.0,790.0,0.1,West,UPI
1002,2026-01-01,8,Notebook,Stationery,5.0,110.0,0.1,West,Credit Card
1003,2026-01-01,36,Pen Set,Stationery,5.0,240.0,0.0,East,Debit Card
1004,2026-01-02,21,Notebook,Stationery,2.0,120.0,0.05,West,UPI
1005,2026-01-03,22,T-Shirt,Clothing,2.0,810.0,0.15,,UPI
1006,2026-01-03,41,T-Shirt,Clothing,5.0,830.0,0.05,South,UPI
1007,2026-01-03,40,Pen Set,Stationery,1.0,260.0,0.15,East,Credit Card
```

customers.csv - 50 rows x 4 columns

customer_id, customer_name, city, segment (Consumer / Corporate / Home Office).

```
customer_id,customer_name,city,segment
1,Vivaan Reddy,Pune,Consumer
2,Neha Mehta,Mumbai,Consumer
3,Arjun Nair,Pune,Corporate
4,Vivaan Patel,Delhi,Corporate
5,Rohan Sharma,Pune,Corporate
```

Task 1: NumPy Introduction & Arrays (10 marks)

Explanation. Imports NumPy and builds a 12-element array, then reports its shape, size and data type with **.shape**, **.size** and **.dtype**. A one-dimensional array is created with **np.arange()** and a 3x4 two-dimensional array from a list of lists; **.ndim** confirms the dimensions.

Source code: task01_numpy_arrays.py

```
# Task 1: NumPy Introduction & Arrays
import numpy as np

# Array with 10+ numbers
arr = np.array([12, 7, 25, 3, 18, 41, 9, 30, 15, 22, 5, 36])
print("Array:", arr)
print("Shape:", arr.shape)
print("Size :", arr.size)
print("Data type:", arr.dtype)

# One-dimensional array
arr_1d = np.arange(1, 11)
print("\n1-D array:", arr_1d)
print("Dimensions:", arr_1d.ndim)

# Two-dimensional array
arr_2d = np.array([[1, 2, 3, 4],
                   [5, 6, 7, 8],
                   [9, 10, 11, 12]])
print("\n2-D array:\n", arr_2d)
print("Dimensions:", arr_2d.ndim, "| Shape:", arr_2d.shape, "| Size:", arr_2d.size, "| dtype:", arr_2d.dtype)
```

Program output

```
Array: [12  7 25  3 18 41  9 30 15 22  5 36]
Shape: (12,)
Size : 12
Data type: int64

1-D array: [ 1  2  3  4  5  6  7  8  9 10]
Dimensions: 1

2-D array:
 [[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Dimensions: 2 | Shape: (3, 4) | Size: 12 | dtype: int64
```

Task 2: NumPy Indexing, Slicing & Reshaping (10 marks)

Explanation. Demonstrates positional indexing (including negative indexes), slicing with **start:stop:step**, and 2-D access of single elements, whole rows, whole columns and sub-matrices. A 12-element array is reshaped into (3,4), (4,3) and (2,2,3); the original is left unchanged because **reshape()** returns a new view.

Source code: task02_indexing_slicing_reshaping.py

```
# Task 2: NumPy Indexing, Slicing & Reshaping
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120])
print("Original 1-D array:", arr)

# Indexing
print("\n--- Indexing ---")
print("First element   arr[0]  :", arr[0])
print("Fifth element   arr[4]  :", arr[4])
print("Last element    arr[-1] :", arr[-1])

# Slicing
print("\n--- Slicing ---")
print("arr[2:7]  :", arr[2:7])
print("arr[:4]   :", arr[:4])
print("arr[::3]  :", arr[::3])
print("arr[::-1] :", arr[::-1])

# 2-D array
arr_2d = np.array([[11, 12, 13, 14],
                   [21, 22, 23, 24],
                   [31, 32, 33, 34]])
print("\n--- 2-D array ---\n", arr_2d)
print("Element at row 1, col 2  arr_2d[1, 2]:", arr_2d[1, 2])
print("Row 0     arr_2d[0]     :", arr_2d[0])
print("Column 1  arr_2d[:, 1]  :", arr_2d[:, 1])
print("Sub-matrix arr_2d[0:2, 1:3]:\n", arr_2d[0:2, 1:3])
# Reshaping
print("\n--- Reshaping ---")
print("Original shape:", arr.shape)
r1 = arr.reshape(3, 4)
print("Reshaped to (3, 4):\n", r1)
r2 = arr.reshape(4, 3)
print("Reshaped to (4, 3):\n", r2)
r3 = arr.reshape(2, 2, 3)
print("Reshaped to (2, 2, 3):\n", r3)
print("Flattened back:", r3.flatten())
```

Program output

```
Original 1-D array: [ 10  20  30  40  50  60  70  80  90 100 110 120]

--- Indexing ---
First element   arr[0]  : 10
Fifth element   arr[4]  : 50
Last element    arr[-1] : 120

--- Slicing ---
arr[2:7]  : [30 40 50 60 70]
arr[:4]   : [10 20 30 40]
arr[::3]  : [ 10  40  70 100]
arr[::-1] : [120 110 100  90  80  70  60  50  40  30  20  10]

--- 2-D array ---
 [[11 12 13 14]
 [21 22 23 24]
 [31 32 33 34]]
Element at row 1, col 2  arr_2d[1, 2]: 23
Row 0     arr_2d[0]     : [11 12 13 14]
Column 1  arr_2d[:, 1]  : [12 22 32]
Sub-matrix arr_2d[0:2, 1:3]:
 [[12 13]
 [22 23]]

--- Reshaping ---
Original shape: (12,)
Reshaped to (3, 4):
 [[ 10  20  30  40]
```

```
 [ 50  60  70  80]
 [ 90 100 110 120]]
Reshaped to (4, 3):
 [[ 10  20  30]
 [ 40  50  60]
 [ 70  80  90]
 [100 110 120]]
Reshaped to (2, 2, 3):
 [[[ 10  20  30]
  [ 40  50  60]]

 [[ 70  80  90]
  [100 110 120]]]
Flattened back: [ 10  20  30  40  50  60  70  80  90 100 110 120]
```

Task 3: NumPy Mathematical & Statistical Operations (10 marks)

Explanation. Uses the marks of 10 students in two subjects. Addition, subtraction, multiplication and division are applied element-wise (vectorised, no loops). Mean, median, min, max, standard deviation and sum are computed for each subject; **axis** is used to get per-subject and per-student means.

Source code: task03_numpy_math_stats.py

```
# Task 3: NumPy Mathematical & Statistical Operations
import numpy as np

# Dataset: marks of 10 students in two subjects
maths   = np.array([78, 85, 92, 67, 74, 88, 95, 60, 81, 70])
science = np.array([72, 80, 89, 70, 68, 91, 90, 65, 77, 75])
print("Maths  :", maths)
print("Science:", science)

print("\n--- Mathematical Operations (element-wise) ---")
print("Addition      :", maths + science)
print("Subtraction   :", maths - science)
print("Multiplication:", maths * science)
print("Division      :", np.round(maths / science, 2))

print("\n--- Statistical Functions (Maths) ---")
print("Mean              :", np.mean(maths))
print("Median            :", np.median(maths))
print("Minimum           :", np.min(maths))
print("Maximum           :", np.max(maths))
print("Standard Deviation:", round(float(np.std(maths)), 2))
print("Sum               :", np.sum(maths))

print("\n--- Statistical Functions (Science) ---")
print("Mean              :", np.mean(science))
print("Median            :", np.median(science))
print("Minimum           :", np.min(science))
print("Maximum           :", np.max(science))
print("Standard Deviation:", round(float(np.std(science)), 2))
print("Sum               :", np.sum(science))

# Bonus: 2-D statistics along axes
marks = np.vstack([maths, science])
print("\nMean per subject (axis=1):", np.mean(marks, axis=1))
print("Mean per student (axis=0):", np.mean(marks, axis=0))
```

Program output

```
Maths  : [78 85 92 67 74 88 95 60 81 70]
Science: [72 80 89 70 68 91 90 65 77 75]

--- Mathematical Operations (element-wise) ---
Addition      : [150 165 181 137 142 179 185 125 158 145]
Subtraction   : [ 6  5  3 -3  6 -3  5 -5  4 -5]
Multiplication: [5616 6800 8188 4690 5032 8008 8550 3900 6237 5250]
Division      : [1.08 1.06 1.03 0.96 1.09 0.97 1.06 0.92 1.05 0.93]

--- Statistical Functions (Maths) ---
Mean              : 79.0
Median            : 79.5
Minimum           : 60
Maximum           : 95
Standard Deviation: 10.76
Sum               : 790

--- Statistical Functions (Science) ---
Mean              : 77.7
Median            : 76.0
Minimum           : 65
Maximum           : 91
Standard Deviation: 9.03
Sum               : 777

Mean per subject (axis=1): [79.  77.7]
Mean per student (axis=0): [75.  82.5 90.5 68.5 71.  89.5 92.5 62.5 79.  72.5]
```

Task 4: Pandas Series & DataFrame (10 marks)

Explanation. Creates a labelled Series of marks and a student DataFrame (Name, Age, Course, Marks). Shows the DataFrame, **.columns** and **.index**, then adds two new columns: **Result** (via apply/lambda) and **Grade** (via **pd.cut**) and displays the updated table.

Source code: task04_pandas_series_dataframe.py

```
# Task 4: Pandas Series & DataFrame
import pandas as pd

# Series
marks = pd.Series([88, 92, 79, 65, 94], index=["Aarav", "Diya", "Rohan", "Isha", "Kabir"], name="Marks")
print("Pandas Series:\n", marks)
print("Series mean:", marks.mean())

# DataFrame - student information
students = pd.DataFrame({
    "Name":   ["Aarav", "Diya", "Rohan", "Isha", "Kabir"],
    "Age":    [20, 21, 20, 22, 21],
    "Course": ["Economics", "Statistics", "Economics", "Commerce", "Statistics"],
    "Marks":  [88, 92, 79, 65, 94],
})
print("\nDataFrame:\n", students)
print("\nColumn names:", list(students.columns))
print("Index       :", students.index)

# Add new column
students["Result"] = students["Marks"].apply(lambda m: "Pass" if m >= 70 else "Needs Improvement")
students["Grade"] = pd.cut(students["Marks"], bins=[0, 69, 79, 89, 100], labels=["D", "C", "B", "A"])
print("\nUpdated DataFrame (with Result & Grade):\n", students)
```

Program output

```
Pandas Series:
 Aarav    88
Diya     92
Rohan    79
Isha     65
Kabir    94
Name: Marks, dtype: int64
Series mean: 83.6

DataFrame:
     Name  Age      Course  Marks
0  Aarav   20   Economics     88
1   Diya   21  Statistics     92
2  Rohan   20   Economics     79
3   Isha   22    Commerce     65
4  Kabir   21  Statistics     94

Column names: ['Name', 'Age', 'Course', 'Marks']
Index       : RangeIndex(start=0, stop=5, step=1)

Updated DataFrame (with Result & Grade):
     Name  Age      Course  Marks            Result Grade
0  Aarav   20   Economics     88              Pass     B
1   Diya   21  Statistics     92              Pass     A
2  Rohan   20   Economics     79              Pass     C
3   Isha   22    Commerce     65  Needs Improvement     D
4  Kabir   21  Statistics     94              Pass     A
```

Task 5: Reading & Inspecting Data (10 marks)

Explanation. Reads **sales_data.csv** (200 orders, 10 columns) with **pd.read_csv()**, then inspects it using head(), tail(), shape, columns, dtypes, info() and describe(). info() already reveals the missing values (non-null counts below 200 in quantity, unit_price, discount and region).

Source code: task05_read_inspect.py

```
# Task 5: Reading & Inspecting Data
import os
import pandas as pd

pd.set_option("display.width", 160)
pd.set_option("display.max_columns", None)
BASE = os.path.dirname(os.path.abspath(__file__))
df = pd.read_csv(os.path.join(BASE, "data", "sales_data.csv"))

print("First 5 rows (head):\n", df.head())
print("\nLast 5 rows (tail):\n", df.tail())
print("\nShape (rows, columns):", df.shape)
print("Number of rows   :", df.shape[0])
print("Number of columns:", df.shape[1])
print("\nColumn names:", list(df.columns))
print("\nData types (dtypes):\n", df.dtypes)
print("\ninfo():")
df.info()
print("\ndescribe():\n", df.describe().round(2))
```

Program output

```
First 5 rows (head):
    order_id  order_date  customer_id   product    category  quantity  unit_price  discount region payment_mode
0      1001  2026-01-01           24   T-Shirt    Clothing       2.0       790.0      0.10   West          UPI
1      1002  2026-01-01            8  Notebook  Stationery       5.0       110.0      0.10   West  Credit Card
2      1003  2026-01-01           36   Pen Set  Stationery       5.0       240.0      0.00   East   Debit Card
3      1004  2026-01-02           21  Notebook  Stationery       2.0       120.0      0.05   West          UPI
4      1005  2026-01-03           22   T-Shirt    Clothing       2.0       810.0      0.15    NaN          UPI

Last 5 rows (tail):
      order_id  order_date  customer_id     product     category  quantity  unit_price  discount region payment_mode
195      1196  2026-06-24           27  Headphones  Electronics       4.0      3020.0       NaN  South          UPI
196      1197  2026-06-27           21    Sneakers     Footwear       4.0      3440.0      0.10   East  Credit Card
197      1198  2026-06-29            6    Notebook   Stationery       3.0       110.0      0.15  North          UPI
198      1199  2026-06-29           27    Sneakers     Footwear       5.0      3670.0      0.00  North         Cash
199      1200  2026-06-29            5     T-Shirt     Clothing       3.0       820.0      0.15   West   Debit Card

Shape (rows, columns): (200, 10)
Number of rows   : 200
Number of columns: 10

Column names: ['order_id', 'order_date', 'customer_id', 'product', 'category', 'quantity', 'unit_price', 'discount', 'region', 'payment_mode']

Data types (dtypes):
 order_id          int64
order_date          str
customer_id       int64
product             str
category            str
quantity        float64
unit_price      float64
discount        float64
region              str
payment_mode        str
dtype: object

info():
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 10 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   order_id      200 non-null    int64
 1   order_date    200 non-null    str
 2   customer_id   200 non-null    int64
 3   product       200 non-null    str
 4   category      200 non-null    str
 5   quantity      188 non-null    float64
 6   unit_price    190 non-null    float64
 7   discount      190 non-null    float64
 8   region        192 non-null    str
 9   payment_mode  200 non-null    str
dtypes: float64(3), int64(2), str(5)
memory usage: 15.8 KB

describe():
        order_id  customer_id  quantity  unit_price  discount
```

```
count    200.00       200.00    188.00       190.00    190.00
mean    1100.50        25.04      3.22      7809.37      0.05
std       57.88        14.12      1.39     14008.77      0.05
min     1001.00         1.00      1.00       110.00      0.00
25%     1050.75        13.00      2.00       782.50      0.00
50%     1100.50        26.00      3.00      1825.00      0.05
75%     1150.25        36.25      4.00      4165.00      0.10
max     1200.00        50.00      5.00     56720.00      0.15
```

Task 6: Selecting, Filtering & Sorting Data (10 marks)

Explanation. Selects columns with **df[[...]]**, rows with **iloc**/**loc**, and filters with boolean conditions: a single condition (unit_price > 20000), multiple AND conditions (&), and **isin()**. Sorting uses **sort_values()** in ascending and descending order, plus a two-column sort.

Source code: task06_select_filter_sort.py

```
# Task 6: Selecting, Filtering & Sorting Data
import os
import pandas as pd

pd.set_option("display.width", 150)
BASE = os.path.dirname(os.path.abspath(__file__))
df = pd.read_csv(os.path.join(BASE, "data", "sales_data.csv"))

print("--- Select specific columns ---")
print(df[["order_id", "product", "quantity", "unit_price"]].head())

print("\n--- Select specific rows ---")
print("df.iloc[5:10]:\n", df.iloc[5:10][["order_id", "product", "region"]])
print("df.loc[0:2, ['order_id','product']]:\n", df.loc[0:2, ["order_id", "product"]])

print("\n--- Filter: unit_price > 20000 ---")
expensive = df[df["unit_price"] > 20000]
print(expensive[["order_id", "product", "unit_price"]].head())
print("Matching rows:", len(expensive))

print("\n--- Multiple conditions: Electronics AND region == 'North' AND quantity >= 3 ---")
multi = df[(df["category"] == "Electronics") & (df["region"] == "North") & (df["quantity"] >= 3)]
print(multi[["order_id", "product", "region", "quantity", "unit_price"]])

print("\n--- OR condition + isin: payment_mode is UPI or Cash ---")
print(df[df["payment_mode"].isin(["UPI", "Cash"])][["order_id", "payment_mode"]].head())

print("\n--- Sort ascending by unit_price (top 5) ---")
print(df.sort_values("unit_price", ascending=True)[["order_id", "product", "unit_price"]].head())

print("\n--- Sort descending by unit_price (top 5) ---")
print(df.sort_values("unit_price", ascending=False)[["order_id", "product", "unit_price"]].head())

print("\n--- Sort by two columns: category (A-Z), then quantity (high-low) ---")
print(df.sort_values(["category", "quantity"], ascending=[True, False])[["category", "product", "quantity"]].head(8))
```

Program output

```
--- Select specific columns ---
   order_id   product  quantity  unit_price
0      1001   T-Shirt       2.0       790.0
1      1002  Notebook       5.0       110.0
2      1003   Pen Set       5.0       240.0
3      1004  Notebook       2.0       120.0
4      1005   T-Shirt       2.0       810.0

--- Select specific rows ---
df.iloc[5:10]:
    order_id   product region
5      1006   T-Shirt  South
6      1007   Pen Set   East
7      1008  Notebook   West
8      1009  Notebook  South
9      1010  Notebook   West
df.loc[0:2, ['order_id','product']]:
    order_id   product
0      1001   T-Shirt
1      1002  Notebook
2      1003   Pen Set

--- Filter: unit_price > 20000 ---
    order_id     product  unit_price
13      1014      Laptop     54530.0
17      1018  Smartphone     22380.0
28      1029  Smartphone     21010.0
31      1032  Smartphone     22950.0
```

```
40      1041      Laptop     53600.0
Matching rows: 38

--- Multiple conditions: Electronics AND region == 'North' AND quantity >= 3 ---
     order_id     product region  quantity  unit_price
10       1011  Headphones  North       3.0      3040.0
23       1024  Headphones  North       3.0         NaN
36       1037  Headphones  North       5.0      3110.0
39       1040  Headphones  North       4.0      3000.0
98       1099  Headphones  North       3.0      3130.0
165      1166  Smartphone  North       4.0     21140.0
187      1188  Smartphone  North       5.0     22780.0
190      1191  Headphones  North       4.0      3110.0

--- OR condition + isin: payment_mode is UPI or Cash ---
   order_id payment_mode
0      1001          UPI
3      1004          UPI
4      1005          UPI
5      1006          UPI
7      1008          UPI

--- Sort ascending by unit_price (top 5) ---
     order_id   product  unit_price
1        1002  Notebook       110.0
102      1103  Notebook       110.0
197      1198  Notebook       110.0
154      1155  Notebook       110.0
9        1010  Notebook       120.0

--- Sort descending by unit_price (top 5) ---
     order_id product  unit_price
173      1174  Laptop     56720.0
75       1076  Laptop     55660.0
81       1082  Laptop     55500.0
13       1014  Laptop     54530.0
185      1186  Laptop     54420.0

--- Sort by two columns: category (A-Z), then quantity (high-low) ---
        category   product  quantity
20   Accessories     Watch       5.0
29   Accessories     Watch       5.0
137  Accessories  Backpack       5.0
177  Accessories  Backpack       5.0
192  Accessories  Backpack       5.0
50   Accessories     Watch       4.0
64   Accessories  Backpack       4.0
77   Accessories  Backpack       4.0
```

Task 7: Handling Missing Values (10 marks)

Explanation. Finds missing values with **isnull()** and counts them per column (40 missing cells across 38 rows). Two approaches are compared: **dropna()** (loses 38 rows, 19% of the data) and **fillna()** with sensible values: median for quantity, the product's average price for unit_price, 0 for discount and the mode for region. Before/after views are shown. **Why it matters:** missing data biases averages and totals, breaks many functions and models, and dropping too much shrinks the sample, so handling it carefully keeps analysis accurate and trustworthy.

Source code: task07_missing_values.py

```
# Task 7: Handling Missing Values
import os
import pandas as pd

pd.set_option("display.width", 160)
pd.set_option("display.max_columns", None)
BASE = os.path.dirname(os.path.abspath(__file__))
df = pd.read_csv(os.path.join(BASE, "data", "sales_data.csv"))

print("=== BEFORE handling missing values ===")
print("Shape:", df.shape)
print("\nisnull() on first 5 rows:\n", df.isnull().head())
print("\nMissing values per column:\n", df.isnull().sum())
print("Total missing cells:", df.isnull().sum().sum())
print("Rows with at least one missing value:", df.isnull().any(axis=1).sum())
print("\nRows containing missing values (sample):\n", df[df.isnull().any(axis=1)].head())

# Method 1: Drop rows
dropped = df.dropna()
print("\n=== Method 1: dropna() ===")
print("Shape after dropping rows:", dropped.shape, "-> rows lost:", len(df) - len(dropped))

# Method 2: Fill with appropriate values
filled = df.copy()
filled["quantity"] = filled["quantity"].fillna(filled["quantity"].median())              # median (count data)
filled["unit_price"] = filled["unit_price"].fillna(                                     # mean price of same product
    filled.groupby("product")["unit_price"].transform("mean").round(-1))
filled["discount"] = filled["discount"].fillna(0)                                       # assume no discount
filled["region"] = filled["region"].fillna(filled["region"].mode()[0])                  # most frequent region

print("\n=== Method 2: fillna() ===")
print("Missing values per column after filling:\n", filled.isnull().sum())
print("Shape after filling:", filled.shape, "(no rows lost)")
print("\nMedian used for quantity :", df["quantity"].median())
print("Mode used for region     :", df["region"].mode()[0])

print("\n=== AFTER: sample of rows that originally had gaps ===")
idx = df[df.isnull().any(axis=1)].index[:5]
print("Before:\n", df.loc[idx, ["order_id", "product", "quantity", "unit_price", "discount", "region"]])
print("After:\n", filled.loc[idx, ["order_id", "product", "quantity", "unit_price", "discount", "region"]])

print("""
Why handling missing data matters in Data Analytics:
 1. Missing values bias statistics (means, totals, correlations) and give wrong insights.
 2. Many functions/ML models fail or silently skip rows containing NaN.
 3. Dropping too much data shrinks the sample; filling sensibly keeps information.
 4. Clean, complete data makes dashboards and reports trustworthy.""")
```

Program output

```
=== BEFORE handling missing values ===
Shape: (200, 10)

isnull() on first 5 rows:
    order_id  order_date  customer_id  product  category  quantity  unit_price  discount  region  payment_mode
0     False       False        False    False     False     False       False     False   False         False
1     False       False        False    False     False     False       False     False   False         False
2     False       False        False    False     False     False       False     False   False         False
3     False       False        False    False     False     False       False     False   False         False
4     False       False        False    False     False     False       False     False    True         False

Missing values per column:
 order_id         0
order_date       0
customer_id      0
product          0
category         0
quantity        12
unit_price      10
discount        10
region           8
payment_mode     0
dtype: int64
Total missing cells: 40
Rows with at least one missing value: 38

Rows containing missing values (sample):
     order_id  order_date  customer_id     product     category  quantity  unit_price  discount region payment_mode
4        1005  2026-01-03           22     T-Shirt     Clothing       2.0       810.0      0.15    NaN          UPI
11       1012  2026-01-07            7       Jeans     Clothing       NaN      1840.0      0.15  North          UPI
20       1021  2026-01-17            3       Watch  Accessories       5.0         NaN      0.00   East          UPI
23       1024  2026-01-18           31  Headphones  Electronics       3.0         NaN       NaN  North  Credit Card
24       1025  2026-01-18            7     T-Shirt     Clothing       3.0       810.0       NaN  South  Credit Card

=== Method 1: dropna() ===
Shape after dropping rows: (162, 10) -> rows lost: 38

=== Method 2: fillna() ===
Missing values per column after filling:
 order_id        0
order_date      0
customer_id     0
product         0
category        0
quantity        0
unit_price      0
discount        0
region          0
payment_mode    0
dtype: int64
Shape after filling: (200, 10) (no rows lost)

Median used for quantity : 3.0
Mode used for region     : South

=== AFTER: sample of rows that originally had gaps ===
Before:
     order_id     product  quantity  unit_price  discount region
4        1005     T-Shirt       2.0       810.0      0.15    NaN
11       1012       Jeans       NaN      1840.0      0.15  North
20       1021       Watch       5.0         NaN      0.00   East
23       1024  Headphones       3.0         NaN       NaN  North
24       1025     T-Shirt       3.0       810.0       NaN  South
After:
     order_id     product  quantity  unit_price  discount region
4        1005     T-Shirt       2.0       810.0      0.15  South
11       1012       Jeans       3.0      1840.0      0.15  North
20       1021       Watch       5.0      4480.0      0.00   East
23       1024  Headphones       3.0      3030.0      0.00  North
24       1025     T-Shirt       3.0       810.0      0.00  South

Why handling missing data matters in Data Analytics:
 1. Missing values bias statistics (means, totals, correlations) and give wrong insights.
 2. Many functions/ML models fail or silently skip rows containing NaN.
 3. Dropping too much data shrinks the sample; filling sensibly keeps information.
 4. Clean, complete data makes dashboards and reports trustworthy.
```

Task 8: Merge, Concatenate, GroupBy & Pivot Table (15 marks)

Explanation. Merges **sales** with **customers** on **customer_id** (left join) to add customer name, city and segment; concatenates DataFrames row-wise and column-wise with **pd.concat()**. **groupby()** produces revenue by category, multi-aggregate summaries by region (sum, mean, count, min, max) and segment averages. **pd.pivot_table()** summarises revenue as category x region (with totals) and average quantity as segment x payment mode.

Source code: task08_merge_groupby_pivot.py

```
# Task 8: Merge, Concatenate, GroupBy & Pivot Table
import os
import pandas as pd

pd.set_option("display.width", 160)
pd.set_option("display.max_columns", None)
BASE = os.path.dirname(os.path.abspath(__file__))
sales = pd.read_csv(os.path.join(BASE, "data", "sales_data.csv"))
customers = pd.read_csv(os.path.join(BASE, "data", "customers.csv"))

# Quick clean so aggregates are meaningful
sales["quantity"] = sales["quantity"].fillna(sales["quantity"].median())
sales["unit_price"] = sales["unit_price"].fillna(sales.groupby("product")["unit_price"].transform("mean"))
sales["discount"] = sales["discount"].fillna(0)
sales["region"] = sales["region"].fillna("Unknown")
sales["revenue"] = sales["quantity"] * sales["unit_price"] * (1 - sales["discount"])

print("=== MERGE: sales + customers on 'customer_id' ===")
merged = pd.merge(sales, customers, on="customer_id", how="left")
print(merged[["order_id", "customer_id", "customer_name", "city", "segment", "product", "revenue"]].head())
print("Shape after merge:", merged.shape)

print("\n=== CONCATENATE: first 100 orders + last 100 orders (stack rows) ===")
part1, part2 = sales.iloc[:100], sales.iloc[100:]
combined = pd.concat([part1, part2], ignore_index=True)
print("part1:", part1.shape, "| part2:", part2.shape, "| combined:", combined.shape)
side = pd.concat([sales[["order_id", "product"]].head(3), sales[["quantity", "revenue"]].head(3)], axis=1)
print("Column-wise concat (axis=1):\n", side)
print("\n=== GROUPBY: revenue by category ===")
print(merged.groupby("category")["revenue"].sum().round(0).sort_values(ascending=False))

print("\n=== GROUPBY: multiple aggregates by region ===")
agg = merged.groupby("region").agg(
    total_revenue=("revenue", "sum"),
    avg_revenue=("revenue", "mean"),
    orders=("order_id", "count"),
    min_qty=("quantity", "min"),
    max_qty=("quantity", "max"),
).round(0)
print(agg)

print("\n=== GROUPBY: average revenue by customer segment ===")
print(merged.groupby("segment")["revenue"].mean().round(0))

print("\n=== PIVOT TABLE: total revenue (rows=category, columns=region) ===")
pivot = pd.pivot_table(merged, values="revenue", index="category", columns="region",
                       aggfunc="sum", fill_value=0, margins=True, margins_name="Total").round(0)
print(pivot)

print("\n=== PIVOT TABLE: avg quantity (rows=segment, columns=payment_mode) ===")
print(pd.pivot_table(merged, values="quantity", index="segment", columns="payment_mode",
                     aggfunc="mean").round(2))
```

Program output

```
=== MERGE: sales + customers on 'customer_id' ===
   order_id  customer_id customer_name       city    segment   product  revenue
0      1001           24    Arjun Nair    Kolkata   Consumer   T-Shirt   1422.0
1      1002            8  Aditya Reddy  Hyderabad  Corporate  Notebook    495.0
2      1003           36   Pooja Verma  Bengaluru   Consumer   Pen Set   1200.0
3      1004           21    Diya Reddy    Kolkata  Corporate  Notebook    228.0
4      1005           22  Aditya Singh  Hyderabad   Consumer   T-Shirt   1377.0
Shape after merge: (200, 14)

=== CONCATENATE: first 100 orders + last 100 orders (stack rows) ===
part1: (100, 11) | part2: (100, 11) | combined: (200, 11)
Column-wise concat (axis=1):
    order_id   product  quantity  revenue
0      1001   T-Shirt       2.0   1422.0
1      1002  Notebook       5.0    495.0
2      1003   Pen Set       5.0   1200.0

=== GROUPBY: revenue by category ===
category
Electronics    4409543.0
Accessories     206450.0
Clothing        193674.0
Footwear        180631.0
Stationery       23562.0
Name: revenue, dtype: float64

=== GROUPBY: multiple aggregates by region ===
         total_revenue  avg_revenue  orders  min_qty  max_qty
region
East         1103099.0      22981.0      48      1.0      5.0
North         454926.0      11373.0      40      1.0      5.0
South        1354416.0      24186.0      56      1.0      5.0
Unknown       184083.0      23010.0       8      1.0      5.0
West         1917335.0      39944.0      48      1.0      5.0

=== GROUPBY: average revenue by customer segment ===
segment
Consumer       30459.0
Corporate      18508.0
Home Office    23289.0
Name: revenue, dtype: float64

=== PIVOT TABLE: total revenue (rows=category, columns=region) ===
region            East     North      South   Unknown       West      Total
category
Accessories    66278.0   14445.0    82082.0   17780.0    25864.0   206450.0
Clothing       55110.0   57027.0    40448.0    5840.0    35250.0   193674.0
Electronics   904197.0  356262.0  1205158.0  146907.0  1797019.0  4409543.0
Footwear       70454.0   21790.0    21528.0   13556.0    53303.0   180631.0
Stationery      7060.0    5402.0     5200.0       0.0     5899.0    23562.0
Total        1103099.0  454926.0  1354416.0  184083.0  1917335.0  5013860.0

=== PIVOT TABLE: avg quantity (rows=segment, columns=payment_mode) ===
payment_mode  Cash  Credit Card  Debit Card   UPI
segment
Consumer      2.75         3.46        3.14  3.33
Corporate     3.17         3.47        3.29  3.03
Home Office   3.50         2.80        3.75  2.55
```

Task 9: Exporting Data (5 marks)

Explanation. Applies the cleaning steps, adds a computed **revenue** column and exports with **to_csv(index=False)** to *output/processed_sales_data.csv*. The file is read back to verify shape, columns, zero missing values and that its values match the in-memory DataFrame.

Source code: task09_export.py

```
# Task 9: Exporting Data
import os
import pandas as pd

BASE = os.path.dirname(os.path.abspath(__file__))
df = pd.read_csv(os.path.join(BASE, "data", "sales_data.csv"))

# Process the dataset
df["quantity"] = df["quantity"].fillna(df["quantity"].median())
df["unit_price"] = df["unit_price"].fillna(df.groupby("product")["unit_price"].transform("mean").round(-1))
df["discount"] = df["discount"].fillna(0)
df["region"] = df["region"].fillna(df["region"].mode()[0])
df["revenue"] = (df["quantity"] * df["unit_price"] * (1 - df["discount"])).round(2)

out_path = os.path.join(BASE, "output", "processed_sales_data.csv")
df.to_csv(out_path, index=False)
print("Exported to:", out_path)

# Verify the exported file
check = pd.read_csv(out_path)
print("\nVerification - re-read exported file:")
print("Shape            :", check.shape)
print("Columns          :", list(check.columns))
print("Missing values   :", int(check.isnull().sum().sum()))
print("Matches in-memory data (values):", bool((check.drop(columns='order_date').round(2).equals(df.drop(columns='order_date').round(2)))))
print("\nFirst 5 rows of exported file:\n", check.head().to_string())
```

Program output

```
Exported to: /home/claude/submission/output/processed_sales_data.csv

Verification - re-read exported file:
Shape            : (200, 11)
Columns          : ['order_id', 'order_date', 'customer_id', 'product', 'category', 'quantity', 'unit_price', 'discount', 'region', 'payment_mode', 'revenue']
Missing values   : 0
Matches in-memory data (values): True

First 5 rows of exported file:
    order_id  order_date  customer_id   product    category  quantity  unit_price  discount region payment_mode  revenue
0      1001  2026-01-01           24   T-Shirt    Clothing       2.0       790.0      0.10   West          UPI   1422.0
1      1002  2026-01-01            8  Notebook  Stationery       5.0       110.0      0.10   West  Credit Card    495.0
2      1003  2026-01-01           36   Pen Set  Stationery       5.0       240.0      0.00   East   Debit Card   1200.0
3      1004  2026-01-02           21  Notebook  Stationery       2.0       120.0      0.05   West          UPI    228.0
4      1005  2026-01-03           22   T-Shirt    Clothing       2.0       810.0      0.15  South          UPI   1377.0
```

Task 10: Mini Data Analysis Project - Sales Performance (10 marks)

Explanation. End-to-end analysis of the sales dataset joined with customer data: load, inspect, handle missing values, engineer features (revenue, month, order size via **np.where**), select/filter, sort, GroupBy analysis, two pivot tables, NumPy statistics (mean, median, std, percentile), and export of the cleaned dataset to *output/cleaned_sales_project.csv*.

Source code: task10_mini_project.py

```
# Task 10: Mini Data Analysis Project - Sales Performance Analysis (NumPy + Pandas)
import os
import numpy as np
import pandas as pd

pd.set_option("display.width", 160)
pd.set_option("display.max_columns", None)
BASE = os.path.dirname(os.path.abspath(__file__))

# 1. LOAD ---------------------------------------------------------------------
sales = pd.read_csv(os.path.join(BASE, "data", "sales_data.csv"), parse_dates=["order_date"])
customers = pd.read_csv(os.path.join(BASE, "data", "customers.csv"))
print("1. LOADED: sales", sales.shape, "| customers", customers.shape)

# 2. INSPECT ------------------------------------------------------------------
print("\n2. INSPECTION")
print(sales.head(3))
print("\nDtypes:\n", sales.dtypes)
print("\nMissing values:\n", sales.isnull().sum()[sales.isnull().sum() > 0])
print("Duplicate order_ids:", sales["order_id"].duplicated().sum())

# 3. HANDLE MISSING VALUES ----------------------------------------------------
sales["quantity"] = sales["quantity"].fillna(sales["quantity"].median())
sales["unit_price"] = sales["unit_price"].fillna(sales.groupby("product")["unit_price"].transform("mean").round(-1))
sales["discount"] = sales["discount"].fillna(0)
sales["region"] = sales["region"].fillna(sales["region"].mode()[0])
print("\n3. After cleaning, total missing values:", int(sales.isnull().sum().sum()))
# Feature engineering (NumPy + Pandas)
sales["revenue"] = np.round(sales["quantity"] * sales["unit_price"] * (1 - sales["discount"]), 2)
sales["month"] = sales["order_date"].dt.month_name()
sales["month_no"] = sales["order_date"].dt.month
sales["order_size"] = np.where(sales["revenue"] >= sales["revenue"].median(), "Large", "Small")
df = sales.merge(customers, on="customer_id", how="left")

# 4. SELECT & FILTER ----------------------------------------------------------
print("\n4. SELECT & FILTER")
high_value = df[df["revenue"] > 100000]
print("Orders with revenue > Rs 1,00,000:", len(high_value))
print(high_value[["order_id", "product", "city", "revenue"]].head())
upi_big = df[(df["payment_mode"] == "UPI") & (df["discount"] >= 0.10)]
print("UPI orders with >=10% discount:", len(upi_big))

# 5. SORT ---------------------------------------------------------------------
print("\n5. TOP 5 ORDERS BY REVENUE")
print(df.sort_values("revenue", ascending=False)[["order_id", "product", "city", "quantity", "revenue"]].head())

# 6. GROUPBY ------------------------------------------------------------------
print("\n6. GROUPBY ANALYSIS")
by_product = df.groupby("product").agg(revenue=("revenue", "sum"), units=("quantity", "sum"),
                                       orders=("order_id", "count")).sort_values("revenue", ascending=False)
by_product["revenue_share_%"] = (by_product["revenue"] / by_product["revenue"].sum() * 100).round(1)
print("Revenue by product:\n", by_product.round(0))
by_cat = df.groupby("category")["revenue"].sum().sort_values(ascending=False)
by_region = df.groupby("region")["revenue"].sum().sort_values(ascending=False)
by_city = df.groupby("city")["revenue"].sum().sort_values(ascending=False)
```

```
by_seg = df.groupby("segment").agg(revenue=("revenue", "sum"), avg_order=("revenue", "mean"))
by_pay = df.groupby("payment_mode")["order_id"].count().sort_values(ascending=False)
by_month = df.groupby(["month_no", "month"])["revenue"].sum()
print("\nRevenue by category:\n", by_cat.round(0))
print("\nRevenue by region:\n", by_region.round(0))
print("\nTop 3 cities:\n", by_city.head(3).round(0))
print("\nSegment summary:\n", by_seg.round(0))
print("\nOrders by payment mode:\n", by_pay)
print("\nMonthly revenue:\n", by_month.round(0))
disc = df.groupby("discount")["quantity"].mean().round(2)
print("\nAvg quantity per order by discount level:\n", disc)

# 7. PIVOT TABLES ------------------------------------------------------------
print("\n7. PIVOT TABLES")
pv1 = pd.pivot_table(df, values="revenue", index="category", columns="region", aggfunc="sum",
                     fill_value=0, margins=True, margins_name="Total").round(0)
print("Revenue: category x region\n", pv1)
pv2 = pd.pivot_table(df, values="revenue", index=["month_no", "month"], columns="category",
                     aggfunc="sum", fill_value=0).round(0)
print("\nRevenue: month x category\n", pv2)

# 8. NUMPY STATISTICS --------------------------------------------------------
rev = df["revenue"].to_numpy()
print("\n8. NUMPY STATS on revenue")
print("Mean:", round(float(np.mean(rev)), 2), "| Median:", round(float(np.median(rev)), 2),
      "| Std:", round(float(np.std(rev)), 2))
print("90th percentile:", round(float(np.percentile(rev, 90)), 2))
top20 = df.sort_values("revenue", ascending=False).head(int(0.2 * len(df)))["revenue"].sum()
print("Top 20% of orders generate", round(top20 / rev.sum() * 100, 1), "% of total revenue")
print("Total revenue:", round(float(rev.sum()), 2))

# 9. EXPORT ------------------------------------------------------------------
out = os.path.join(BASE, "output", "cleaned_sales_project.csv")
df.drop(columns=["month_no"]).to_csv(out, index=False)
print("\n9. EXPORTED cleaned dataset ->", out, "| shape:", df.drop(columns=['month_no']).shape)
```

Program output

```
1. LOADED: sales (200, 10) | customers (50, 4)

2. INSPECTION
   order_id order_date  customer_id   product    category  quantity  unit_price  discount region payment_mode
0      1001 2026-01-01           24   T-Shirt    Clothing       2.0       790.0       0.1   West          UPI
1      1002 2026-01-01            8  Notebook  Stationery       5.0       110.0       0.1   West  Credit Card
2      1003 2026-01-01           36   Pen Set  Stationery       5.0       240.0       0.0   East   Debit Card

Dtypes:
 order_id                int64
order_date      datetime64[us]
customer_id              int64
product                    str
category                   str
quantity               float64
unit_price             float64
discount               float64
region                     str
payment_mode               str
dtype: object

Missing values:
 quantity      12
unit_price    10
discount      10
region         8
dtype: int64
Duplicate order_ids: 0
```

```
3. After cleaning, total missing values: 0

4. SELECT & FILTER
Orders with revenue > Rs 1,00,000: 16
    order_id product     city   revenue
13      1014  Laptop    Delhi  147231.0
40      1041  Laptop  Chennai  160800.0
47      1048  Laptop     Pune  200830.0
61      1062  Laptop  Chennai  214320.0
75      1076  Laptop    Delhi  166980.0
UPI orders with >=10% discount: 31

5. TOP 5 ORDERS BY REVENUE
     order_id product     city  quantity   revenue
81       1082  Laptop   Mumbai       5.0  277500.0
152      1153  Laptop    Delhi       5.0  228012.5
173      1174  Laptop  Kolkata       4.0  226880.0
61       1062  Laptop  Chennai       4.0  214320.0
47       1048  Laptop     Pune       4.0  200830.0

6. GROUPBY ANALYSIS
Revenue by product:
              revenue  units  orders  revenue_share_%
product
Laptop      2438124.0   48.0      13             49.0
Smartphone  1751540.0   83.0      27             35.0
Headphones   219902.0   78.0      26              4.0
Sneakers     180652.0   55.0      17              4.0
Watch        140891.0   33.0      11              3.0
Jeans        135346.0   79.0      22              3.0
Backpack      65578.0   46.0      13              1.0
T-Shirt       58328.0   77.0      28              1.0
Pen Set       14026.0   58.0      17              0.0
Notebook       9546.0   84.0      26              0.0

Revenue by category:
 category
Electronics    4409566.0
Accessories     206470.0
Clothing        193674.0
Footwear        180652.0
Stationery       23572.0
Name: revenue, dtype: float64

Revenue by region:
 region
West     1917358.0
South    1538498.0
East     1103136.0
North     454942.0
Name: revenue, dtype: float64

Top 3 cities:
 city
Pune       1396894.0
Chennai    1143524.0
Delhi       931974.0
Name: revenue, dtype: float64

Segment summary:
               revenue  avg_order
segment
Consumer     3076425.0    30460.0
Corporate    1425126.0    18508.0
Home Office   512384.0    23290.0

Orders by payment mode:
 payment_mode
UPI            94
Credit Card    50
Debit Card     40
Cash           16
Name: order_id, dtype: int64

Monthly revenue:
 month_no  month
1         January      668684.0
2         February     570999.0
3         March       1081816.0
4         April       1137366.0
5         May          812546.0
6         June         742524.0
Name: revenue, dtype: float64
```

```
Avg quantity per order by discount level:
 discount
0.00    3.15
0.05    3.17
0.10    3.32
0.15    3.30
Name: quantity, dtype: float64

7. PIVOT TABLES
Revenue: category x region
 region            East     North      South       West      Total
category
Accessories    66288.0   14455.0    99862.0    25864.0   206470.0
Clothing       55110.0   57027.0    46288.0    35250.0   193674.0
Electronics   904203.0  356262.0  1352065.0  1797036.0  4409566.0
Footwear       70475.0   21790.0    35084.0    53303.0   180652.0
Stationery      7060.0    5408.0     5200.0     5904.0    23572.0
Total        1103136.0  454942.0  1538498.0  1917358.0  5013935.0

Revenue: month x category
 category           Accessories  Clothing  Electronics  Footwear  Stationery
month_no month
1        January       60096.0   48060.0     538560.0   15273.0      6696.0
2        February      23914.0   32736.0     489994.0   21397.0      2958.0
3        March         10560.0   11540.0    1013339.0   44396.0      1980.0
4        April         24127.0   50640.0    1034069.0   25666.0      2864.0
5        May           27280.0   33467.0     729460.0   16256.0      6083.0
6        June          60492.0   17231.0     604146.0   57664.0      2992.0

8. NUMPY STATS on revenue
Mean: 25069.67 | Median: 5720.0 | Std: 49779.2
90th percentile: 86720.0
Top 20% of orders generate 83.6 % of total revenue
Total revenue: 5013935.0

9. EXPORTED cleaned dataset -> /home/claude/submission/output/cleaned_sales_project.csv | shape: (200, 16)
```

Key Findings from the Mini Data Analysis Project

Based on 200 orders worth Rs 50.14 lakh in total revenue (January - June 2026) after cleaning:

Electronics dominates revenue. Electronics brings in about 88% of revenue (Rs 44.10 lakh). Laptops alone contribute 49% from just 13 orders, and Smartphones add another 35%.

Revenue is highly concentrated. The top 20% of orders generate 83.6% of total revenue. The mean order value (Rs 25,070) is more than four times the median (Rs 5,720), a strongly right-skewed distribution driven by a few big-ticket laptop orders.

West and South lead regionally; North lags. West (Rs 19.2 lakh, 38%) and South (Rs 15.4 lakh, 31%) account for roughly 70% of sales, while North contributes only about 9%, a clear growth opportunity.

Pune, Chennai and Delhi are the top cities. Pune alone generates about 28% of revenue (Rs 13.97 lakh), followed by Chennai (Rs 11.44 lakh) and Delhi (Rs 9.32 lakh).

Sales peak in March-April. April is the best month (Rs 11.37 lakh) and March close behind (Rs 10.82 lakh), while February is the weakest (Rs 5.71 lakh). Electronics drives these swings.

Consumers are the biggest segment. The Consumer segment contributes about 61% of revenue and also has the highest average order value (Rs 30,460) versus Corporate (Rs 18,508).

UPI is the preferred payment method. UPI accounts for 94 of 200 orders (47%), followed by Credit Card (25%), Debit Card (20%) and Cash (8%).

Discounts barely change basket size. Average quantity per order only moves from 3.15 (no discount) to about 3.3 (10-15% discount), so deeper discounts do not appear to drive much extra volume.

Recommendations

- Protect and grow the Electronics line (stock availability, bundles) since it carries most of the revenue.
- Run targeted campaigns in the North region and for Corporate customers to broaden the revenue base.
- Review blanket discounts: they cost margin without a visible lift in quantity.
- Plan inventory and promotions around the March-April peak and look for ways to lift February.

Note: the dataset is synthetically generated for practice, so these findings illustrate the analysis workflow rather than real business results.

Files Submitted (Google Drive)

```
Week2_Assignment/
 |-- task01_numpy_arrays.py ... task10_mini_project.py   (10 scripts)
 |-- make_datasets.py                                    (generates the CSV data)
 |-- data/
 |     |-- sales_data.csv        (input, contains missing values)
 |     `-- customers.csv         (input)
 `-- output/
       |-- processed_sales_data.csv   (Task 9 export)
       |-- cleaned_sales_project.csv  (Task 10 export)
       `-- taskNN_*.txt               (captured outputs)
```